

Apache Spark Core Fundamentals

1. Introduction to Distributed Computing

Single-Machine Computing vs. Distributed Computing

In traditional single-machine computing, data processing is constrained by the physical hardware of a single server (CPU cores, RAM, and local disk capacity). When datasets scale into terabytes or petabytes, scaling vertically (upgrading to a larger machine) becomes economically prohibitive and physically impossible due to hardware limitations.

Distributed Computing links multiple independent computers (nodes) together over a high-speed network to operate as a single unified system (a cluster).

Core Challenges Solved by Apache Spark

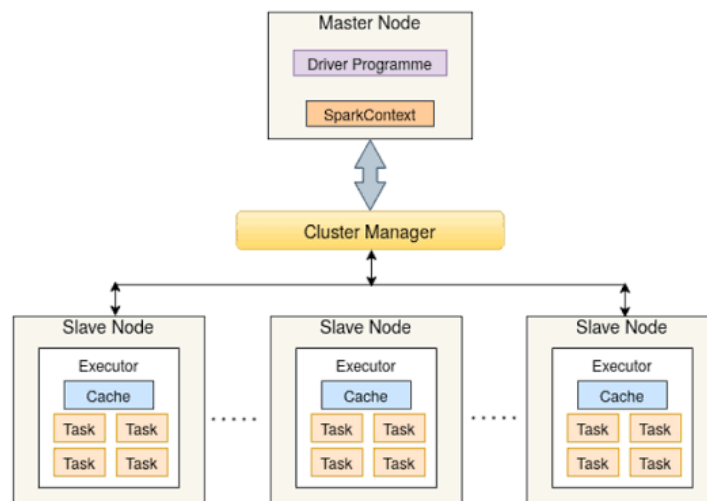
Distributed systems face three major architecture challenges:

1. **Data Division:** Splitting large files into uniform data chunks distributed across nodes.
2. **Task Coordination:** Orchestrating computation tasks to process local data chunks in parallel.
3. **Fault Tolerance:** Automatically recovering when individual hardware nodes fail during job execution without losing completed computation progress.

Apache Spark handles data division, task scheduling, network shuffling, and automatic failure recovery behind high-level abstraction layers.

2. Spark Cluster Architecture: Driver & Executors

Apache Spark uses a master-worker distributed architecture managed by a **Cluster Manager**.



The Driver Node

The **Driver Node** is the master process that controls the lifecycle of a Spark application.

- **Responsibilities:**
 - Initializes the `SparkSession` (the entry point for Spark operations).
 - Converts user code into physical operations by generating a **Directed Acyclic Graph (DAG)**.
 - Breaks DAGs down into **Stages** and discrete **Tasks**.
 - Requests compute resources from the Cluster Manager.
 - Schedules and distributes tasks across active Executor processes.
 - Collects final summary results (e.g., when calling actions like `.collect()` or `.count()`).

Operational Nuance: If the Driver Node crashes, the entire Spark application fails immediately, regardless of worker health.

The Cluster Manager

The **Cluster Manager** allocates physical resources (CPU, RAM) across the cluster. Spark supports multiple cluster managers:

- **Standalone:** Built-in cluster manager included with Spark.
- **YARN:** Standard resource manager in Hadoop ecosystems.
- **Kubernetes (K8s):** Container-native orchestration platform.

Worker Nodes & Executor Processes

- **Worker Nodes:** Physical or virtual machines running in the cloud or on-premises that host executor processes.
- **Executor Processes:** JVM processes launched on worker nodes to execute computational tasks and store cached data in memory or disk.
- **Task Slots (Execution Threads):** Each executor process is allocated a fixed number of CPU cores. Each core represents a task slot capable of processing exactly **one data partition** at a time.

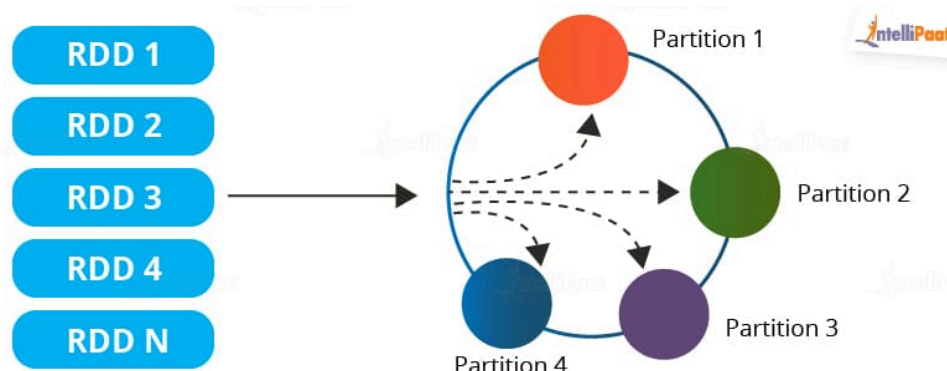
Total Parallel Capacity = (Number of Executors) × (Cores per Executor)³. Resilient Distributed Datasets (RDDs)

The **Resilient Distributed Dataset (RDD)** is Spark's fundamental low-level data abstraction. An RDD is a read-only, partitioned collection of records distributed across nodes in the cluster.

Key Characteristics of RDDs

1. **Resilient (Fault-Tolerant):** If a worker node holding an RDD partition fails, Spark does not need to recompute the entire dataset from scratch. Instead, it uses the **Lineage Graph** (the history of transformation steps used to build the RDD) to recompute only the lost partition.
2. **Distributed:** The dataset is split across memory and disk across multiple nodes in the cluster.

3. **Immutable:** Once created, an RDD cannot be modified in place. Any transformation operation produces a brand-new RDD.



Creating RDDs in PySpark

Create RDD from local collection

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
rdd_from_list = spark.sparkContext.parallelize(data, numSlices=4)
```

Create RDD from external text file

```
rdd_from_file =
```

```
spark.sparkContext.textFile("abfss://data@storage.dfs.core.windows.net/raw_logs.txt")
```

When to Use RDDs vs. High-Level APIs

Scenario / Requirement	Use RDDs	Use DataFrames / Datasets
Unstructured text, binary data, or low-level custom serialization	Yes	No
Structured or semi-structured data (CSV, JSON, Parquet, Delta)	No	Yes
Require automated query optimizations (Catalyst Optimizer)	No	Yes
Need precise functional programming controls over domain objects	Yes	No

4. DataFrames, Datasets & Engine Optimizations

A **DataFrame** is a distributed collection of data organized into named columns, conceptually equivalent to a table in a relational database or a data frame in R/Pandas, but built with optimized distributed execution under the hood.

The Data Abstraction Evolution

- **RDDs:** Untyped/Generic Java objects; no explicit schema awareness; no engine query optimization.
- **DataFrames:** Schema-aware collections optimized using Catalyst and Tungsten; available across Python, Scala, Java, and R.
- **Datasets:** Strongly-typed API combining JVM type safety with Catalyst optimizations. Available **only in Scala and Java** (Python is dynamically typed and uses DataFrames).

Core Optimization Engines

DataFrames outperform raw RDDs due to two internal optimization engines:

1. Catalyst Optimizer

The Catalyst Optimizer transforms high-level DataFrame code through four steps:

1. **Unresolved Logical Plan:** Parses SQL or DataFrame operations to construct a basic Abstract Syntax Tree (AST).
2. **Resolved Logical Plan:** Cross-references table and column names against the system catalog to verify schemas.
3. **Optimized Logical Plan:** Applies rule-based optimization patterns, including:
 - **Predicate Pushdown:** Moves `.filter()` constraints as close to the storage layer as possible so irrelevant data is pruned before reading over the network.
 - **Projection Pruning:** Reads only required columns, skipping unreferenced data attributes.
4. **Physical Plan Generation:** Evaluates multiple physical execution strategies (e.g., choosing a Broadcast Hash Join vs. Sort-Merge Join) and generates optimized Java bytecode.

2. Tungsten Engine

The Tungsten Engine focuses on hardware-level memory and CPU optimization:

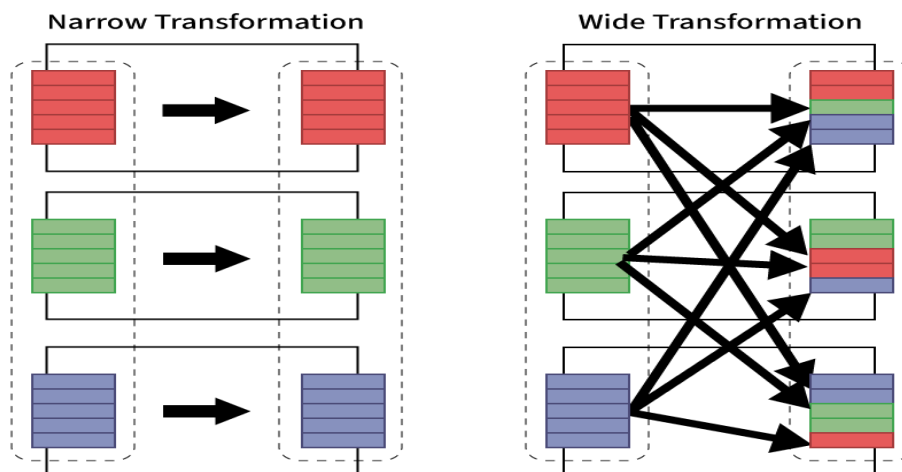
- **Off-Heap Memory Management:** Bypasses standard JVM garbage collection overhead by managing memory explicitly in binary format using `sun.misc.Unsafe`.

- **Whole-Stage Code Generation:** Fuses multiple physical operators (e.g., filter, project, aggregate) into a single optimized Java function to optimize CPU register utilization.

5. Transformations: Narrow vs. Wide

Transformations are operations on a DataFrame or RDD that return a new dataset rather than updating data in place. They are split into **Narrow Transformations** and **Wide Transformations**.

Wide & Narrow dependencies in Apache Spark



1. Narrow Transformations

In a **Narrow Transformation**, all data required to calculate the results for a single output partition resides in a single input partition on the same node. **No network data movement (shuffling) is required.**

- **Characteristics:** Executed in parallel within local worker thread pipelines.
- **Common Examples:** `.filter()`, `.map()`, `.flatMap()`, `.select()`, `.withColumn()`, `.drop()`.

Narrow Transformation Example (Local pipeline processing)

```
df_filtered = df_orders.filter(df_orders["status"] == "COMPLETED") \
    .select("order_id", "customer_id", "total_amount")
```

2. Wide Transformations

In a **Wide Transformation**, computing a single output partition requires reading data from multiple input partitions scattered across different cluster nodes. **This forces a Shuffle Operation.**

- **Characteristics:** Spark must write data to local disk, transfer records across the network, and group records on recipient nodes. Shuffles introduce network IO and disk serialization

latency.

- **Common Examples:** `.groupBy()`, `.join()`, `.distinct()`, `.reduceByKey()`, `.repartition()`, `.orderBy()`.

Wide Transformation Example (Triggers network shuffle to group by customer)

```
df_grouped = df_orders.groupBy("customer_id") \
    .agg({"total_amount": "sum"})
```

6. Actions & Execution Triggers

An **Action** triggers execution of the compiled physical plan to return a result to the Driver program or write data out to an external storage location (e.g., ADLS, S3, or PostgreSQL).

Common Actions in PySpark

Actions that return data to the Driver Node memory:

```
count_val = df.count()          # Returns total row count as Integer
collected_rows = df.collect()   # Driver gathers ALL rows into local memory (Risk of OOM!)
first_row = df.first()           # Returns the first Row object
head_rows = df.take(5)           # Returns an array of the first N Row objects
```

Actions that write data out to Storage Sinks:

```
df.write.format("parquet").save("abfss://gold@storage.dfs.core.windows.net/sales")
df.write.format("delta").mode("overwrite").saveAsTable("gold.sales_summary")
```

Memory Safety Alert: Calling `.collect()` brings the entire distributed dataset onto the single Driver Node. If the dataset size exceeds the Driver Node's available RAM, the application will crash with an `OutOfMemoryError (OOM)`. Use `.take(n)` or `.limit(n)` for debugging instead.

7. Lazy Evaluation & DAG Generation

How Lazy Evaluation Works

Spark uses **Lazy Evaluation**, meaning that when you invoke transformations on a `DataFrame`, Spark does not execute the computations immediately. Instead, it records these steps as a logical lineage inside a **Directed Acyclic Graph (DAG)**.

Computation occurs **only when an Action is called**.

Practical Benefits of Lazy Evaluation

1. **Global Query Optimization:** Because Spark sees the entire chain of transformations before execution, the Catalyst Optimizer can rearrange operations, prune unused columns,

and combine pipeline stages.

2. **Lineage-Based Fault Tolerance:** If a worker node crashes mid-pipeline, Spark uses the DAG lineage to re-execute only the missing partition operations on a healthy node.
3. **Elimination of Unnecessary Processing:** If a user defines ¹⁰ sequential filter steps followed by `.take(5)`, Spark reads only enough data records to produce ⁵ matching output rows, rather than processing millions of rows through all ¹⁰ steps first.

8. Partitioning Strategies & Data Skew Optimization

A **Partition** is a logical, contiguous slice of data distributed across the cluster. Partitioning is Spark's fundamental unit of parallelism—each executor thread processes exactly **one partition at a time**.

Dataset (10 GB File)

└─ Partition 1 (2 GB)	─► Executed on Worker 1 (Core 1)
└─ Partition 2 (2 GB)	─► Executed on Worker 1 (Core 2)
└─ Partition 3 (2 GB)	─► Executed on Worker 2 (Core 1)
└─ Partition 4 (2 GB)	─► Executed on Worker 2 (Core 2)
└─ Partition 5 (2 GB)	─► Executed on Worker 3 (Core 1)

Recommended Partition Sizing

- **Target Partition Size:** A good rule of thumb is aiming for **100 MB – 200 MB per partition** in memory.
- **Under-partitioning:** Partitions are too large (>1gb). Causes memory pressure, disk spilling, and idle CPU cores.
- **Over-partitioning:** Too many small partitions (<10mb). Generates excessive task scheduling and RPC coordinator overhead.

Resizing Partitions: `repartition()` vs. `coalesce()`

Spark provides two primary functions for modifying dataset partition counts:

1. `repartition(n)` - FULL SHUFFLE

Increases or decreases the number of partitions. Redistributes data uniformly using a full network shuffle.

```
df_repartitioned = df.repartition(20)
```

2. `coalesce(n)` - MINIMIZED SHUFFLE

Decreases the number of partitions by merging adjacent partitions on the same node without full network shuffles.

```
df_coalesced = df.coalesce(2)
```

Feature / Dimension	repartition(n)	coalesce(n)
Primary Purpose	Increase or decrease partition count	Decrease partition count only
Network Shuffle	Always triggers a full network shuffle	Avoids full shuffle (merges local partitions)
Data Balance	Produces evenly balanced partition sizes	May create unevenly sized partitions
Common Use Case	Balancing data before heavy joins or aggregations	Combining small output files before writing to storage

Managing Data Skew

Data Skew occurs when data is distributed unevenly across partitions. For example, if 90% of sales records belong to a single null key or high-volume region (region_id = 'US'), one executor task will take hours to finish while all other executors sit idle.

Balanced Partitions (Ideal):

Task 1: [100 MB] [Done in 2 mins]

Task 2: [100 MB] [Done in 2 mins]

Task 3: [100 MB] [Done in 2 mins]

Bottleneck

Data Skew (Bottleneck):

Task 1: [10 MB] [Done in 10 secs]

Task 2: [10 MB] [Done in 10 secs]

Task 3: [1.8 GB] [Processing... 2 Hours!] ←

Techniques to Resolve Data Skew

1. **Salting:** Append a random integer suffix (key_0, key_1, key_2) to high-frequency join keys to distribute skewed key values across multiple partitions, performing the join and stripping the salt afterwards.
2. **Broadcast Joins:** When joining a large skewed table with a small table, use a Broadcast Join to bypass table shuffling entirely:

```
from pyspark.sql.functions import broadcast
```

```
# Broadcasts small dimension table to all executors, eliminating the join shuffle step
df_joined = df_large_fact.join(broadcast(df_small_dim), "category_id")
```